

Introduction to Natural Language Processing

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026

Natural Language Processing



1. Motivation
2. Learning Outcomes
3. Introduction
4. Classical Text Classification
5. N-grams
6. Sequence Notation
7. Probabilistic Notation
8. Count and Probability Matrices
9. Probability of a Sequence
10. Start and End Tokens
11. Out of vocabulary words
12. Evaluating Language Models: Perplexity
13. Limitations

14. Summary

15. References

- ▶ Language helps us talk to each other—and now, to computers too!
- ▶ NLP (Natural Language Processing) teaches machines to understand, interpret, and generate human language.
- ▶ Why is NLP important?
 - Makes **chatbots** like ChatGPT and Siri possible
 - Powers **search engines** like Google
 - Helps **translate languages** (Google Translate)
 - Finds out what **people feel in reviews** (sentiment analysis)
- ▶ NLP is foundational to advanced AI systems.

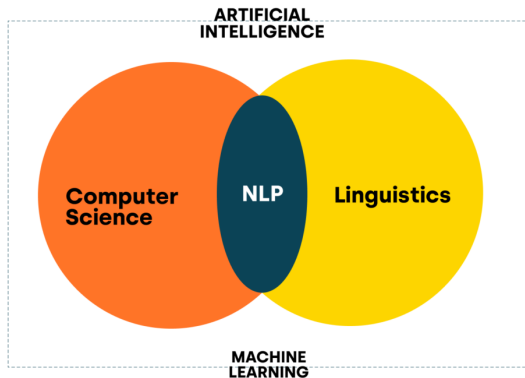
- ▶ **Define** Natural Language Processing (NLP)
- ▶ **Explain and construct** N-grams (unigram, bigram, trigram)
- ▶ **Understand** sequence notation and tokenization
- ▶ **Compute** N-gram probabilities and count matrices
- ▶ **Apply** start/end tokens and handle unknown words (OOV, UNK)
- ▶ **Recognize** the limitations of N-gram models and future directions

Natural Language Processing: **Introduction**

What is NLP?

- ▶ Study of computational approaches to processing natural languages.
- ▶ Processing includes:
 - Acquiring language data
 - Representing information
 - Storing text and speech
 - Understanding meaning
 - Characterizing language patterns
 - Generating new language
- ▶ Natural languages refer to human languages.

What is Natural Language Processing?



NLP = Computer Science + Linguistics + AI

► Deals with:

- Language understanding (input)
- Language generation (output)
- Language translation
- Information extraction

► Subfields:

- Syntax
- Semantics
- Pragmatics
- Discourse



Goal: Deep Understanding

Requires context, linguistic structure, meanings...



To Avoid: Shallow Matching

Could be useful also though depending on use case

Goal of NLP:

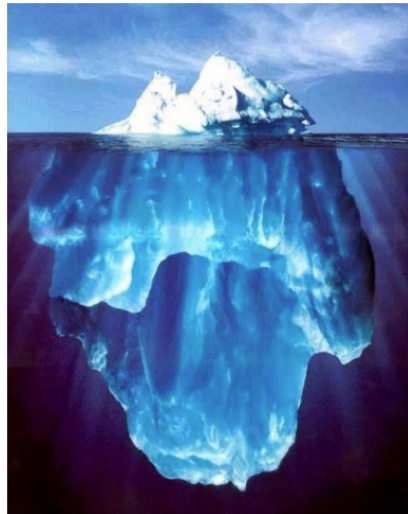
- ▶ Enable machines to understand and generate human language.
- ▶ Facilitate human-computer interaction through natural language.
- ▶ Develop systems that can process and analyze large amounts of text data.

- ▶ **Text Preprocessing**
 - Cleaning and preparing raw text for analysis.
- ▶ **Tokenization**
 - Splitting text into words, sentences, or other meaningful units.
- ▶ **POS Tagging**
 - Assigning parts of speech (noun, verb, etc.) to each token.
- ▶ **Parsing**
 - Analyzing grammatical structure of sentences.
- ▶ **Named Entity Recognition (NER)**
 - Identifying entities such as people, organizations, locations.
- ▶ **Sentiment Analysis / Classification**
 - Determining sentiment or categorizing text.
- ▶ **Language Modeling**
 - Predicting the next word or sequence in text.

- ▶ An iceberg is a large piece of freshwater ice that has broken off from a snow-formed glacier or ice shelf and is floating in open water.



- ▶ An iceberg is a large piece of freshwater ice that has broken off from a snow-formed glacier or ice shelf and is floating in open water.



Teaching machines to **understand and generate human language**

► **Text preprocessing:**

Cleaning and preparing raw text data (removing noise, tokenization, normalization).

► **Understanding meaning:**

Extracting meaning from text using techniques like part-of-speech tagging, named entity recognition, and sentiment analysis.

► **Language modeling:**

Building models that can predict or generate text, such as autocomplete or next-word prediction.

► **Translation, summarization, etc.:**

Enabling applications like machine translation, text summarization, question answering, and more.

Text data is everywhere: tweets, reviews, articles, chats! NLP helps us make sense of this vast information.

Task	What It Does	Example
Tokenization	Split text into words	"I love NLP" → ["I", "love", "NLP"]
POS Tagging	Label grammar tags	"Dogs bark" → [Noun, Verb]
Named Entity Recognition	Find names, places, etc.	"Christopher Nolan lives in Los Angeles"
Sentiment Analysis	Detect mood	"This movie was amazing!" → Positive
Machine Translation	Language to language	English → French

► **Lowercase everything:**

Example: "NLP" → "nlp"

► **Removing stop words:**

Eliminate common words that carry little meaning (e.g., "is", "the", "and") to focus on important content.

► **Stemming/Lemmatization:**

Reduce words to their root or base form.

Example: "running" → "run"

► **Vectorization:**

Transform words or documents into numerical representations for machine learning models:

- **Bag of Words:** Counts word occurrences in a document.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** Weighs words by importance across documents.
- **Word2Vec:** Learns dense vector representations capturing word meaning and context.



What is a corpus?

- ▶ A corpus is a collection of text.
- ▶ Often annotated in some way.
- ▶ Sometimes just lots of text.
- ▶ **Balanced corpora:** Usually not possible in practice.
- ▶ **Examples:**
 - Newswire collections: 500M+ words
 - Brown corpus: 1M words of tagged “balanced” text
 - Penn Treebank: 1M words of parsed WSJ
 - Canadian Hansards: 10M+ words of aligned French/English sentences
 - The Web: billions of words of who knows what

- ▶ **Vocabulary** = All unique words in your dataset
- ▶ Example: "I love NLP and NLP loves me" →
Vocabulary = {"I", "love", "NLP", "and", "loves", "me"}
- ▶ More data = Bigger vocabulary = Harder to process!

💡 Tip: Rare words may not help; common words may not mean much.

Traditional approach: One-hot encoding

- ▶ Example: "NLP" $\rightarrow [0, 0, 1, 0, 0, 0, 0, \dots]$

Problem:

- ▶ High-dimensional (thousands of words!)
- ▶ Sparse (mostly 0s)
- ▶ No meaning in structure (no relation between "king" and "queen")
- ▶ ✗ Not efficient for learning

Feature extraction = Turning text into numbers

- ▶ Count how often each word appears (**Term Frequency**)

Example:

Text	"great product"	"bad product"
Word: "great"	1	0
Word: "bad"	0	1

 Use this to find patterns in sentiment, spam, etc.

Suppose you're classifying reviews:

Positive Reviews: ["amazing", "good", "great"]

Negative Reviews: ["bad", "awful", "terrible"]

Count how often each word appears in each class.

Example table:

Word	Positive Count	Negative Count
good	20	1
bad	1	30

Helps models detect the “tone” (**sentiment clues**) of new text.

- ▶ **Bag of Words (BoW)**: Just counts word frequencies in each document.
- ▶ **TF-IDF (Term Frequency-Inverse Document Frequency)**: Adjusts for how “unique” or important a word is in a document compared to all documents.
 - Words like "the", "is" are less important.
- ▶ **Word Embeddings (later)**: Add meaning and capture relationships between words (e.g., similarity, analogy).

NLP: Classical Text Classification

Bayes' Rule:

$$P(\text{Class} \mid \text{Words}) = \frac{P(\text{Words} \mid \text{Class}) \cdot P(\text{Class})}{P(\text{Words})}$$

- ▶ Predict class (e.g., spam or not) given the words
- ▶ Simple, powerful tool used in Naive Bayes classifiers
- ▶ Helps models reason with uncertainty

Simple, fast probabilistic model

Assumes:

- ▶ Words are independent given the class (a “naive” assumption)

Example:

- ▶ "This movie is awesome" $\rightarrow P(\text{Positive})$ high
- ▶ "This movie is boring" $\rightarrow P(\text{Negative})$ high

✓ Works well with BoW, TF-IDF, and N-grams

We're already familiar with probability-based classifiers, and have seen logistic regression applied in computer vision tasks.

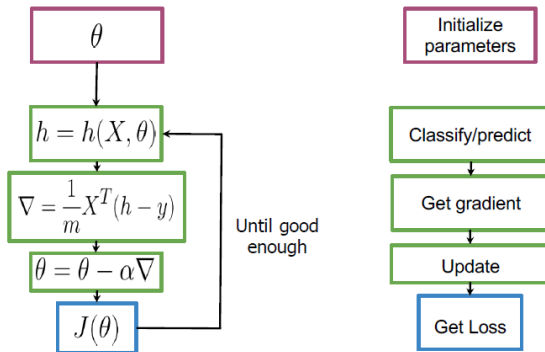
- ▶ Here, logistic regression takes word features (from BoW or TF-IDF)
- ▶ It learns weights to predict if text belongs to a certain class, such as:
 - Positive or Negative
 - Spam or Not Spam
- ▶ Outputs a probability:

$$P(\text{Class} = 1 \mid \text{features}) = \sigma(w_1x_1 + w_2x_2 + \dots + b)$$

- ▶ Words like “awesome” get high weights for positive class

🎯 Intuition:

- ▶ Each word contributes a “vote” toward a class.
- ▶ The model learns which words are most important for classification.



Training a Logistic Regression Model

Natural Language Processing: **N-grams**

What are N-grams?

An N-gram is a sequence of N words

Corpus: I am happy because I am learning

Unigrams: { I , am , happy , because , learning }

Bigrams: { I am , am happy , happy because ... }

✗ I happy

Trigrams: { I am happy , am happy because , ... }

- Items are typically **words** or **characters**.

- ▶ Tokenization is the process of breaking text into smaller units called **tokens**.
- ▶ Tokens can be words, characters, or subwords.
- ▶ Example: "I love NLP" can be tokenized into:
 - Words: ["I", "love", "NLP"]
 - Characters: ["I", " ", "l", "o", "v", "e", " ", "N", "L", "P"]
 - Subwords: ["I", " ", "lov", "e", " ", "N", "L", "P"]
- ▶ Tokenization is crucial for preparing text data for NLP tasks.

Sentence: “I love NLP”

- ▶ **Unigrams:** I, love, NLP
- ▶ **Bigrams:** I love, love NLP
- ▶ **Trigrams:** I love NLP

- ▶ Capture local word co-occurrence
- ▶ Build simple language models
- ▶ Easy to compute and analyze
- ▶ Trade-off between simplicity (unigram) and contextual richness (trigram)

Sequence Notation Basics

Sentence: $w_1, w_2, \dots, w_n$

For example: $w_1 = \text{I}$, $w_2 = \text{love}$, $w_3 = \text{NLP}$

General representation:

- ▶ Unigram: $P(w_i)$
- ▶ Bigram: $P(w_i \mid w_{i-1})$
- ▶ Trigram: $P(w_i \mid w_{i-2}, w_{i-1})$

Sequence Notation Example:

Corpus: This is great ... teacher drinks tea. $m = 500$

$w_1 \ w_2 \ w_3$ $w_{498} \ w_{499} \ w_{500}$

$$w_1^m = w_1 \ w_2 \ \dots \ w_m$$

$$w_1^3 = w_1 \ w_2 \ w_3$$

$$w_{m-2}^m = w_{m-2} \ w_{m-1} \ w_m$$

Sequence Notation Basics

Sentence: $w_1, w_2, \dots, w_n$

For example: $w_1 = \text{I}$, $w_2 = \text{love}$, $w_3 = \text{NLP}$

General representation:

- ▶ Unigram: $P(w_i)$
- ▶ Bigram: $P(w_i \mid w_{i-1})$
- ▶ Trigram: $P(w_i \mid w_{i-2}, w_{i-1})$

Corpus: I am happy because I am learning

Size of corpus $m = 7$

$$P(I) = \frac{2}{7}$$

$$P(happy) = \frac{1}{7}$$

Probability of unigram:

$$P(w) = \frac{C(w)}{m}$$

Corpus: I am happy because I am learning

$$P(am|I) = \frac{C(I \text{ } am)}{C(I)} = \frac{2}{2} = 1$$

$$P(happy|I) = \frac{C(I \text{ } happy)}{C(I)} = \frac{0}{2} = 0 \quad \times \text{ I happy}$$

$$P(learning|am) = \frac{C(am \text{ } learning)}{C(am)} = \frac{1}{2}$$

Probability of a bigram:
$$P(y|x) = \frac{C(x \text{ } y)}{\sum_w C(x \text{ } w)} = \frac{C(x \text{ } y)}{C(x)}$$

Corpus: I am happy because I am learning

$$P(\text{happy} | \text{I am}) = \frac{C(\text{I am happy})}{C(\text{I am})} = \frac{1}{2}$$

Probability of a trigram: $P(w_3 | w_1^2) = \frac{C(w_1^2 w_3)}{C(w_1^2)}$

$$C(w_1^2 w_3) = C(w_1 w_2 w_3) = C(w_1^3)$$

Probability of N-gram: $P(w_N | w_1^{N-1}) = \frac{C(w_1^{N-1} w_N)}{C(w_1^{N-1})}$

$$C(w_1^{N-1} w_N) = C(w_1^N)$$

Objective: Compute the probability of a sentence

N-gram Assumption: The probability of a word depends only on the previous $(n - 1)$ words.

Formula:

$$P(w_1^n) \approx \prod_{i=1}^n P(w_i \mid w_{i-n+1}^{i-1})$$

where w_1^n denotes the sequence $w_1, w_2, \dots, w_n$ and w_{i-n+1}^{i-1} is the context of the previous $(n - 1)$ words.

Bigram MLE:

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

- ▶ $\text{Count}(w_{i-1}, w_i)$: Number of times the bigram (w_{i-1}, w_i) appears in the corpus.
- ▶ $\text{Count}(w_{i-1})$: Number of times the word w_{i-1} appears as a context.

Text: “I love NLP. I love AI.”

Bigrams and Counts:

- ▶ (I, love): 2
- ▶ (love, NLP): 1
- ▶ (love, AI): 1

Probability Calculations:

- ▶ $P(\text{love} \mid \text{I}) = \frac{2}{2} = 1.0$
- ▶ $P(\text{NLP} \mid \text{love}) = \frac{1}{2} = 0.5$
- ▶ $P(\text{AI} \mid \text{love}) = \frac{1}{2} = 0.5$

Count Matrix: A matrix that counts occurrences of word pairs in a corpus.

Probability Matrix: A matrix that calculates probabilities of word pairs based on counts.

Example: For the sentence "I love NLP. I love AI."

- ▶ Count Matrix: Counts how many times each word appears with every other word.
- ▶ Probability Matrix: Calculates the probability of each word appearing given the previous word.

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

- Rows: unique corpus (N-1)-grams
- Columns: unique corpus words

Corpus: <s>I study I learn</s>

- Bigram count matrix

"study I" bigram

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	1	1
study	0	0	1	0	0
learn	0	1	0	0	0

Text: "I love NLP. I love AI."

Bigrams:

- ▶ (I, love): 2
- ▶ (love, NLP): 1
- ▶ (love, AI): 1

Count Matrix:

- ▶ Rows: Words in the corpus
- ▶ Columns: Words in the corpus
- ▶ Cells: Count of occurrences of each word pair

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

- Divide each cell by its row sum

Corpus: `<s>I study I learn</s>`

Count matrix (bigram)

	<s>	</s>	I	study	learn	sum
<s>	0	0	1	0	0	1
</s>	0	0	0	0	0	0
I	0	0	0	1	1	2
study	0	0	1	0	0	1
learn	0	1	0	0	0	1



Probability matrix

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

$$sum(row) = \sum_{w \in V} C(w_{n-N+1}^{n-1}, w) = C(w_{n-N+1}^{n-1})$$

Probability Calculation:

Bigram Probability:

$$P(\text{love} \mid \text{I}) = \frac{\text{Count}(\text{I}, \text{love})}{\text{Count}(\text{I})} = \frac{2}{2} = 1$$

Probability Matrix:

- ▶ Rows: Words in the corpus
- ▶ Columns: Words in the corpus
- ▶ Cells: Probability of each word given the previous word

- Given a sentence, what is its probability?

$$P(\textit{the teacher drinks tea}) = ?$$

- Conditional probability and chain rule reminder

$$P(B|A) = \frac{P(A, B)}{P(A)} \implies P(A, B) = P(A)P(B|A)$$

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

$$P(\textit{the teacher drinks tea}) =$$

$$P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{the teacher}) \\ P(\textit{tea}|\textit{the teacher drinks})$$

Problem: Corpus almost never contains the exact sentence we're interested in or even its longer subsequences!

$$P(\textit{the teacher drinks tea}) =$$

$$P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{the teacher}) \\ P(\textit{tea}|\textit{the teacher drinks})$$

the teacher drinks tea

$$P(\text{tea}|\text{the teacher drinks}) \approx P(\text{tea}|\text{drinks})$$

$$\begin{aligned} &P(\text{teacher}|\text{the}) \\ &P(\text{drinks}|\text{teacher}) \\ &P(\text{tea}|\text{drinks}) \end{aligned}$$

$$\begin{aligned} &P(\text{the teacher drinks tea}) = \\ &P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})\boxed{P(\text{tea}|\text{the teacher drinks})} \\ &\quad \downarrow \\ &P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})\boxed{P(\text{tea}|\text{drinks})} \end{aligned}$$

- Markov assumption: only last N words matter

- Bigram $P(w_n | w_1^{n-1}) \approx P(w_n | w_{n-1})$

- N-gram $P(w_n | w_1^{n-1}) \approx P(w_n | w_{n-N+1}^{n-1})$

- Entire sentence modeled with bigram $P(w_1^n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$

$$P(w_1^n) \approx \boxed{P(w_1)} P(w_2 | w_1) \dots P(w_n | w_{n-1})$$

Objective: Apply sequence probability approximation with bigrams.

Question:

Given these conditional probabilities

$P(\text{Mary})=0.1;$ $P(\text{likes})=0.2;$ $P(\text{cats})=0.3$
 $P(\text{Mary}|\text{likes})=0.2;$ $P(\text{likes}|\text{Mary})=0.3;$ $P(\text{cats}|\text{likes})=0.1;$ $P(\text{likes}|\text{cats})=0.4$

Approximate the probability of the following sentence with bigrams: "Mary likes cats"

Type: Multiple Choice, single answer

Options and solution:

1. $P(\text{Mary likes cats}) = 0$

2. $P(\text{Mary likes cats}) = 1$

3. $P(\text{Mary likes cats}) = 0.003$

4. $P(\text{Mary likes cats}) = 0.008$

N-gram Models: **Start and End Tokens**

- ▶ **Start:** <s> or <start>
- ▶ **End:** </s> or <end>

Benefits:

- ▶ Helps model sentence boundaries
- ▶ Enables generation and evaluation

the teacher drinks tea

$$P(\text{the teacher drinks tea}) \approx P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})P(\text{tea}|\text{drinks})$$



<s> the teacher drinks tea

$$P(\text{<s> the teacher drinks tea}) \approx P(\text{the}|\text{<s>})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})P(\text{tea}|\text{drinks})$$

- Trigram:

$$P(\text{the teacher drinks tea}) \approx$$

$$P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{teacher drinks})$$

the teacher drinks tea $\Rightarrow$ <s> <s> the teacher drinks tea

$$P(w_1^n) \approx P(w_1|\text{<s> <s>})P(w_2|\text{<s> } w_1) \dots P(w_n|w_{n-2} w_{n-1})$$

- N-gram model: add N-1 start tokens <s>

$$P(y|x) = \frac{C(x \ y)}{\sum_w C(x \ w)} = \frac{C(x \ y)}{C(x)}$$

Corpus:

<s> Lyn drinks chocolate

<s> John drinks

$$\sum_w C(drinks \ w) = 1$$

$$C(drinks) = 2$$

Corpus

<s> yes no

<s> yes yes

<s> no no

Sentences of length 2:

<s> yes yes

<s> yes no

<s> no no

<s> no yes

$$P(< s > \text{ yes yes}) =$$

$$P(\text{yes} \mid < s >) \times P(\text{yes} \mid \text{yes}) =$$

$$\frac{C(< s > \text{ yes})}{\sum_w C(< s > w)} \times \frac{C(\text{yes yes})}{\sum_w C(\text{yes } w)} =$$

$$\frac{2}{3} \times \frac{1}{2} = \frac{1}{3}$$

Corpus

$\langle s \rangle$ yes no

$\langle s \rangle$ yes yes

$\langle s \rangle$ no no

Sentences of length 2:

$\langle s \rangle$ yes yes

$\langle s \rangle$ yes no

$\langle s \rangle$ no no

$\langle s \rangle$ no yes

$$P(\langle s \rangle \text{ yes yes}) = \frac{1}{3}$$

$$P(\langle s \rangle \text{ yes no}) = \frac{1}{3}$$

$$P(\langle s \rangle \text{ no no}) = \frac{1}{3}$$

$$P(\langle s \rangle \text{ no yes}) = 0$$

$$\sum_{2 \text{ word}} P(\dots) = 1$$

Corpus

$\langle s \rangle$ yes no

$\langle s \rangle$ yes yes

$\langle s \rangle$ no no

Sentences of length 3:

$\langle s \rangle$ yes yes yes

$\langle s \rangle$ yes yes no

...

$\langle s \rangle$ no no no

$$P(\langle s \rangle \text{ yes yes yes}) = \dots$$

$$P(\langle s \rangle \text{ yes yes no}) = \dots$$

$$\dots = \dots$$

$$P(\langle s \rangle \text{ no no no}) = \dots$$

$$\sum_{\text{3 word}} P(\dots) = 1$$

Corpus

<s> yes no

<s> yes yes

<s> no no

$$\sum_{\text{2 word}} P(\dots) + \sum_{\text{3 word}} P(\dots) + \dots = 1$$

- Bigram

$\langle s \rangle$ the teacher drinks tea $\Rightarrow$ $\langle s \rangle$ the teacher drinks tea $\langle /s \rangle$

$$P(the|\langle s \rangle)P(teacher|the)P(drinks|teacher)P(tea|drinks)P(\langle /s \rangle|tea)$$

Corpus:

$\langle s \rangle$ Lyn drinks chocolate $\langle /s \rangle$

$\langle s \rangle$ John drinks $\langle /s \rangle$

$$\sum_w C(drinks\ w) = 2$$
$$C(drinks) = 2$$

- N-gram => just one </s>

E.g. Trigram:

the teacher drinks tea => <s> <s> the teacher drinks tea </s>

Corpus

<s> Lyn drinks chocolate </s>
<s> John drinks tea </s>

<s> Lyn eats chocolate </s>

$$P(\text{John}|\text{<s>}) = \frac{1}{3}$$

$$P(\text{chocolate}|\text{eats}) = \frac{1}{2}$$

$$P(\text{sentence}) = \frac{2}{3} * \frac{1}{2} * \frac{1}{2} * \frac{2}{2} = \frac{1}{6}$$

$$P(\text{</s>}|\text{tea}) = \frac{1}{1}$$

$$P(\text{Lyn}|\text{<s>}) = ? = \frac{2}{3}$$

Objective: Apply sequence probability approximation with bigrams after adding start and end word.

Question:

Given these conditional probabilities

$$P(\text{Mary})=0.1; \quad P(\text{likes})=0.2; \quad P(\text{cats})=0.3$$

$$P(\text{Mary}|\langle s \rangle)=0.2; \quad P(\langle s \rangle|\text{cats})=0.6$$

$$P(\text{likes}|\text{Mary})=0.3; \quad P(\text{cats}|\text{likes})=0.1$$

Approximate the probability of the following sentence with bigrams: " $\langle s \rangle$ Mary likes cats $\langle s \rangle$ "

Type: Multiple Choice, single answer

Options and solution:

1. $P(\langle s \rangle \text{ Mary likes cats } \langle s \rangle) = 0$

2. $P(\langle s \rangle \text{ Mary likes cats } \langle s \rangle) = 0.0036$

3. $P(\langle s \rangle \text{ Mary likes cats } \langle s \rangle) = 0.003$

4. $P(\langle s \rangle \text{ Mary likes cats } \langle s \rangle) = 1$

N-gram Models: **Out of vocabulary words**

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word "quokka" might not be in the training data.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word "quokka" might not be in the training data.

Impact: OOV words can lead to poor model performance and inaccurate predictions.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word "quokka" might not be in the training data.

Impact: OOV words can lead to poor model performance and inaccurate predictions.

Closed vs. Open Vocabularies:

- ▶ **Closed vocabulary:** Only words seen during training are recognized.
- ▶ **Open vocabulary:** Model can handle unseen words.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word "quokka" might not be in the training data.

Impact: OOV words can lead to poor model performance and inaccurate predictions.

Closed vs. Open Vocabularies:

- ▶ **Closed vocabulary:** Only words seen during training are recognized.
- ▶ **Open vocabulary:** Model can handle unseen words.

Solution: Use a special tag <UNK> in the corpus and input to represent unknown words.

Using <UNK>: Replace rare or unseen words with the special token <UNK>.

Using <UNK>: Replace rare or unseen words with the special token <UNK>.

Why? Helps the model generalize to words it has not seen during training.

Using <UNK>: Replace rare or unseen words with the special token <UNK>.

Why? Helps the model generalize to words it has not seen during training.

Example:

- ▶ Original: I love ChatGPT
- ▶ With OOV handling: I love <UNK>

1. **Create vocabulary V :** Build a list of all words to be recognized (e.g., most frequent words).
2. **Replace OOV words:** For any word in the corpus not in V , replace it with <UNK>.
3. **Estimate probabilities:** Treat <UNK> as a regular word when computing word probabilities.

This approach allows the model to handle unseen words gracefully during inference.

Corpus

<s> Lyn drinks chocolate </s>
<s> John drinks tea </s>
<s> Lyn eats chocolate </s>



Corpus

<s> Lyn drinks chocolate </s>
<s> <UNK> drinks <UNK> </s>
<s> Lyn <UNK> chocolate </s>

Min frequency $f=2$

Vocabulary

Lyn, drinks, chocolate

Input query

<s> Adam drinks chocolate </s>
↓
<s> <UNK> drinks chocolate </s>

Criteria for Vocabulary Selection:

- ▶ **Minimum word frequency f :** Only include words that appear at least f times in the corpus.
- ▶ **Maximum vocabulary size $|V|$:** Limit V to the top $|V|$ most frequent words.

Use <UNK> Sparingly: Choose f and $|V|$ to minimize the number of words replaced by <UNK>, while keeping the vocabulary manageable. **Perplexity:** Only compare language models that use the same vocabulary V to ensure fair evaluation.

- ▶ Perplexity (PPL) measures how well a language model predicts a sequence of words. Intuitively, it reflects “on average, how many choices the model is unsure about” per word.
- ▶ Formula:

$$\text{PPL}(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

- ▶ How to interpret:
 - Lower perplexity $\rightarrow$ model assigns higher probability to the test set sequences $\rightarrow$ better predictions.

► Why it matters:

- Standard metric for evaluating language models, from n-gram models to neural LMs.
- Only compare perplexities of models that share the same vocabulary V .

► Example:

- Bigram model on a test set $\rightarrow$ PPL = 200 (poor).
- Transformer LM $\rightarrow$ PPL = 20 (much better).

N-gram Models: **Limitations**

- ⚠ **Data Sparsity**
- ⚠ **Limited Context** (only $N-1$ words)
- ⚠ **Explodes with Vocabulary Size**
- ⚠ **Doesn't Capture Semantics/Syntax**

Beyond N-grams, NLP as a whole still faces some big challenges:

- ▶ **Bias:** Models can pick up and even increase unfairness from the data they learn from.
- ▶ **Needs Lots of Data:** Modern NLP needs huge amounts of labeled text, which is hard and expensive to get.
- ▶ **Ambiguity:** Language can be confusing! For example, “I saw the man with the telescope”—who has the telescope?
- ▶ **Many Languages:** It’s tough to make models that work well for every language, especially those with little data.
- ▶ **Expensive to Train:** Training big models costs a lot of money and energy, so not everyone can do it.

N-gram Models: **Summary**

- ▶ Neural Language Models (e.g., Word2Vec, LSTMs)
- ▶ Transformer-based Models (BERT, GPT)
- ▶ Subword Tokenization (Byte-Pair Encoding)
- ▶ Pretrained Language Models
- ▶ Contextual Representations

- ▶ NLP enables machines to understand human language
- ▶ N-grams model word sequences simply and effectively
- ▶ Sequence notation and probability estimation are essential
- ▶ Start/end/UNK tokens improve modeling and robustness
- ▶ N-gram models are limited → neural models offer solutions

NLP: References

- ▶ Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing (3rd Ed Draft)* – <https://web.stanford.edu/~jurafsky/slp3/>
- ▶ CMU CS 11-711 Advanced NLP (Graham Neubig) – <https://www.phontron.com/class/anlp2024/>
- ▶ Stanford CS224N (2023) – <https://web.stanford.edu/class/cs224n/>
- ▶ Manning, C. D., et al. (2008). *Introduction to Information Retrieval*
- ▶ Bengio et al. (2003) – A Neural Probabilistic Language Model, JMLR
- ▶ Mikolov et al. (2013) – Efficient Estimation of Word Representations in Vector Space
- ▶ Vaswani et al. (2017) – Attention is All You Need
- ▶ Younes Mourri & Lukasz Kaiser, *Natural Language Processing Specialization*, DeepLearning.AI – <https://www.deeplearning.ai/courses/natural-language-processing-specialization/>